INTRODUCTION TO PROGRAMMING
REFERENCE SHEET

HEADER FILES

```
#include <stdio.h>      // printf, scanf, getchar, putchar
#include <stdlib.h>     // abs, fabs
#include <string.h>     // strlen, strcpy, strcmp, strcat
#include <math.h>       // sqrt, pow, ceil, floor
#include <ctype.h>      // isdigit, isalpha, toupper, tolower
#include <stdbool.h>    // bool, true, false
```

VARIABLES AND DATA TYPES

```
int age = 20;
float marks = 95.5f;
double value = 3.14159;
char grade = 'A';
```

Common data types:

`char`	character
`int`	integer
`float`	single-precision decimal
`double`	double-precision decimal
`long long`	large integer
`bool`	true / false

NOTE:

```
Character -> 'A'
String    -> "ABC"
String    -> Char array
```

CONSTANTS

```
const int MAX = 100;
const float PI = 3.14159;

#define MAX 100
#define PI 3.14159
```

INPUT AND OUTPUT

Format Specifiers

%d	int
%u	unsigned int
%ld	long
%lld	long long
%f	float
%lf	double
%c	char
%s	string
%x	hexadecimal
%o	octal
%p	pointer
%zu	sizeof result

IMPORTANT:

In `scanf`, the input format is very important.

```
// Input: x y z
scanf("%d %d %d", &x, &y, &z);

// Input: x,y,z
scanf("%d,%d,%d", &x, &y, &z);

// If input is in two lines:
// hello (string)
// w (char)

scanf("%s", str);
scanf(" %c", &ch);
```

The space before %c skips whitespace/newline. Without the space the input may get corrupted.

```
scanf("%d", &x);      // & required for normal variables
scanf("%s", str);     // no & for character array/string
```

CHARACTER & STRING INPUT / OUTPUT

```
char ch;

ch = getchar();
putchar(ch);

char str[100];
scanf(" %s", str);        //reads till first whitespace.
scanf(" %99[^\n]", str);  //reads strings with spaces.

-- Reads everything until a newline '\n' is encountered.
-- The 99 to prevent the array from overflowing.
```

OPERATOR PRECEDENCE

Higher precedence generally comes first:

1. () []
2. ! ++ -
3. * / %
4. + -
5. < <= > >=
6. == !=
7. &
8. ^
9. |
10. &&
11. ||
12. ?:
13. = += -= *= /= %=
14. ,

When in doubt, use parentheses.

```
if ((a > b) && (b > c)) {
    // code
}
```

INTEGER DIVISION

```
5 / 2          -> 2     (int)
5.0 / 2        -> 2.5
(float)5 / 2   -> 2.5
```

IMPORTANT: Integer / Integer → Integer

TYPE CASTING

```
float result = (float)a / b;

int x = (int)3.14;      // 3

double d = (double)x;
double d = (double)32;  //32.00
```

IF / ELSE

```
if (condition) {
    // code
}
else {
    // code
}

Example:
if (marks >= 90) {
    printf("A");
}
else if (marks >= 80) {
    printf("B");
}
else {
```

```
    printf("C");
}
```

TERNARY OPERATOR

```
condition ? value_if_true : value_if_false;

int max = (a > b) ? a : b;
```

If / Else in a single line.

SWITCH

```
switch (choice) {

    case 1:
        printf("One");
        break;

    case 2:
        printf("Two");
        break;

    default:
        printf("Invalid");
}
```

IMPORTANT: `break` prevents execution from falling into the next case.

FOR LOOP

```
for (initialization; condition; update) {
    // code
}

for (int i = 0; i < n; i++) {
    printf("%d ", i);
}
```

WHILE LOOP

```
while (condition) {
    // code
}
```

DO-WHILE LOOP

```
do {
    // code
} while (condition);
```

IMPORTANT: `do-while` executes at least once, because it runs the code and then checks the while condition.

BREAK AND CONTINUE

```
break;
    // Immediately exits the loop/switch.

continue;
    // Skips the current iteration.

Example:

for (int i = 0; i < n; i++) {

    if (i == 5)
        break;

    if (i % 2 == 0)
        continue;

    printf("%d ", i);
}
```

Because of `break`, the loop ends when `i == 5`.

Because of `continue`, the loop skips the next part after the if statement whenever `(i % 2 == 0)`.

ARRAYS

```
int arr[100];

int arr[] = {10, 20, 30, 40};
```

IMPORTANT:

Array indexing starts from 0.

For an array of size n: valid indices are from 0 to $n - 1$

2D ARRAYS

```
int matrix[3][4];
int matrix[2][3] = {{1, 2, 3},{4, 5, 6}};
```

STRINGS

A string is a character array ending with `'\0'`.

```
char str[] = "Hello";

char str[] = {'H', 'e', 'l', 'l', 'o', '\0'};

char str[100];
```

IMPORTANT:

Always leave space for `'\0'`.

If the number of characters in a word is 5, create a character array of size 6 : char arr[6];

STRING FUNCTIONS

```
#include <string.h>

strlen(str);                      // Returns string length (excluding '\0')

strcpy(destination, source);     // Copies source into destination

strcmp(str1, str2);              // 0 if strings are equal

strcat(str1, str2);              // Appends str2 to str1
```

Example:

```
char a[100];
char b[] = "Hello";

strcpy(a, b);
```

IMPORTANT: Destination must have enough space.

FUNCTIONS

**Function Declaration (No func code or implementation) → Main Function → Function Definition (implementation)

or

** Function Definition → Main Function

Function declaration:

```
return_type function_name(parameters);

int add(int a, int b);
```

Function definition:

```
int add(int a, int b) {
    return a + b;
}
```

Function call:

```
int result = add(10, 20);
```

Function with no return value:

```
void display() {
    printf("Hello");
}
```

FUNCTIONS WITH ARRAYS

1D Array:

```
int func(int arr[], int size) {
    // function code
}
```

.

2D Array:

cols should be initialized or defined before.

```
int func(int arr[][cols], int rows) {
    // function code
}

OR

int func(int arr[rows][cols], int rows) {
    // function code
}
```

For arrays, the array name is enough in the function call. No need for brackets [] or dimensions.

```
int array[5] = {1, 2, 3, 4, 5};

int value = func(array, 5);
```

MATH FUNCTIONS

```
#include <math.h>

sqrt(x)       -> Square root
pow(x, y)     -> x raised to y
ceil(x)       -> Round upward
floor(x)      -> Round downward
fabs(x)       -> Absolute value of floating-point number

For integer absolute value:

abs(x);

#include <stdlib.h>
```

CHARACTER FUNCTIONS

```
#include <ctype.h>

isalpha(ch)   -> Alphabet?
isdigit(ch)   -> Digit?
isalnum(ch)   -> Alphabet or digit?
isspace(ch)   -> Whitespace?
islower(ch)   -> Lowercase?
isupper(ch)   -> Uppercase?

tolower(ch)   -> Convert to lowercase
toupper(ch)   -> Convert to uppercase
```

Using ASCII Values

```
ASCII RANGES (No need to remember)
'A' to 'Z'  -> 65 to 90
'a' to 'z'  -> 97 to 122
'0' to '9'  -> 48 to 57

isalpha(ch)
------------------------------------------------------------
Checks whether ch is an alphabet.

if ((ch >= 'A' && ch <= 'Z') || (ch >= 'a' && ch <= 'z')) {
    // Alphabet
}

isdigit(ch)
------------------------------------------------------------
Checks whether ch is a digit.

if (ch >= '0' && ch <= '9') {
    // Digit
}
```

```
isalnum(ch)
------------------------------------------------------------
Checks whether ch is an alphabet OR digit.

if ((ch >= 'A' && ch <= 'Z') || (ch >= 'a' && ch <= 'z') || (ch >= '0' && ch <= '9')) {
    // Alphabet or digit
}

isspace(ch)
------------------------------------------------------------
Common whitespace characters in ASCII:
' '   -> Space | '\t'  -> Tab | '\n'  -> Newline | '\r'  -> Carriage return | '\v'  -> Vertical tab | '\f'  -> Form feed

Check:
if (ch == ' '  || ch == '\t' || ch == '\n' || ch == '\r' || ch == '\v' || ch == '\f') {
    // Whitespace
}


islower(ch)
------------------------------------------------------------
Checks whether ch is a lowercase alphabet.

if (ch >= 'a' && ch <= 'z') {
    // Lowercase
}


isupper(ch)
------------------------------------------------------------
Checks whether ch is an uppercase alphabet.

if (ch >= 'A' && ch <= 'Z') {
    // Uppercase
}


tolower(ch)
------------------------------------------------------------
Convert uppercase alphabet to lowercase.

if (ch >= 'A' && ch <= 'Z') {
    ch = 'a' + (ch - 'A');
}


toupper(ch)
------------------------------------------------------------
Convert lowercase alphabet to uppercase.

if (ch >= 'a' && ch <= 'z') {
    ch = 'A' + (ch - 'a');
}
```

COMMON NUMBER OPERATIONS

Reverse a number:

```
//input is n
int rev = 0;

while (n > 0) {
    int digit = n % 10;
    rev = rev * 10 + digit;
    n /= 10;
}
```

Count digits:

```
int count = 0;

while (n > 0) {
    count++;
    n /= 10;
}
```

Sum of digits:

```
int sum = 0;

while (n > 0) {
    sum += n % 10;
```

```
    n /= 10;
}
```

SWAP TWO VARIABLES

```
int temp = a;
a = b;
b = temp;
```

FINDING MAXIMUM/MINIMUM IN ARRAY

```
int max = arr[0];

for (int i = 1; i < n; i++) {

    if (arr[i] > max)
        max = arr[i];
}
```

.

```
int min = arr[0];

for (int i = 1; i < n; i++) {

    if (arr[i] < min)
        min = arr[i];
}
```

LINEAR SEARCH

```
int found = 0;

for (int i = 0; i < n; i++) {

    if (arr[i] == key) {
        found = 1;
        break;
    }
}

if (found == 0) {
    printf("Not found");
}
```

REVERSE AN ARRAY

```
int i = 0;
int j = n - 1;

while (i < j) {

    int temp = arr[i];
    arr[i] = arr[j];
    arr[j] = temp;

    i++;
    j--;
}
```

GCD (Greatest Common Divisor)

```
int gcd(int a, int b) {

    while (b != 0) {

        int temp = b;
        b = a % b;
        a = temp;
    }

    return a;
}
```

.

IMPORTANT RULES TO REMEMBER

- Array indexing starts from 0.
- For an array of size n: valid indices are from 0 to $n-1$
- Strings end with `'\0'`.
- `scanf()` generally requires the address of a variable.
- Use == for comparison.
- Use = for assignment.
- Integer / integer gives integer division.
- Always initialize variables before using them, because without initialization they may contain garbage values.
- Check array bounds.
- `switch` cases generally require `break`.
- Use parentheses when operator precedence is unclear.
- Always make sure loops can terminate.
- Make sure strings have enough space for `\0`.

DEBUGGING METHODS

Important: Learn to read error messages and identify errors from them. Some of the Compilation Errors

1. Missing headers or Missing function defination or Incorrect function name

```
check.c: In function 'main':
check.c:6:41: error: implicit declaration of function 'abs' [-Wimplicit-function-declaration]
    6 |     printf("The value of |x| is: %d\n", abs(x));
      |                                        |~~
```

Implicit Declaration -> Either you have missed some header related to the function or The function definition is not there.

2. Variable not defined or Spelling mistake

```
check.c: In function 'main':
check.c:7:41: error: 'numbber' undeclared (first use in this function); did you mean 'number'?
    7 |     printf("The value of |x| is: %d\n", numbber);
      |                                        ^~~~~~~
      |                                        number
```

.

3. Missing Semicolon or Bracket

```
check.c: In function 'main':
check.c:8:5: error: expected ',' or ';' before 'if'
    8 |     if(x < y) {
      |     ^~
check.c:10:7: error: 'else' without a previous 'if'
   10 |     } else {
      |       ^~~~
check.c:13:1: error: expected declaration or statement at end of input
   13 | }
      | ^
```

You can see the line numbers of errors and what is missing.

4. Wrong function parameters

```
check.c: In function 'main':
check.c:8:19: error: too few arguments to function 'add'; expected 2, have 1
    8 |    printf("%d\n", add(5));
      |                   ^~~
check.c:3:5: note: declared here
    3 | int add(int a, int b) {
      |     ^~~
```

Incorrect number of parameters or the datatype is wrong.

Runtime Errors

1. The main reason : **Out of bounds Array Access**.
Example: int arr[5]; then accessing arr[5] or arr[6]
2. Division by zero: Handle Manually